

MÉTRICAS DE CALIDAD

EcoRoute — Densidad de Defectos y Complejidad Ciclomática

1. INTRODUCCIÓN

1.1 Propósito

Este documento define, mide y analiza las métricas de calidad del código fuente del sistema EcoRoute. Las métricas cubren dos dimensiones principales: la densidad de defectos (cuántos bugs existen por unidad de código) y la complejidad ciclomática (qué tan complejas son las rutas de ejecución del código). Ambas métricas son predictores confiables de la mantenibilidad y la probabilidad de defectos futuros.

2. MÉTRICAS DE COMPLEJIDAD CICLOMÁTICA

2.1 Definición

La complejidad ciclomática (CC) de McCabe mide el número de rutas de ejecución independientes en una función o método. Se calcula como:

$CC = \text{Número de decisiones binarias} + 1$

Cada if, elif, for, while, except, and, or incrementa la CC en 1.

2.2 Escala de Interpretación

Complejidad Ciclomática	Clasificación	Riesgo	Acción recomendada
1 — 5	Baja	Bajo	Ninguna
6 — 10	Moderada	Medio	Aceptable, monitorear
11 — 15	Alta	Alto	Refactorizar en próxima iteración
16 — 20	Muy alta	Muy alto	Refactorizar en el sprint actual
> 20	Extrema	Crítico	Refactorizar inmediatamente

2.3 Resultados por Función — Módulos Críticos

Completar tras ejecutar: `radon cc . -a -s --min B`

`environment/sumo_env.py`

Función / Método	CC	Clasificación	Acción
SumoEnv.step()	—	—	Pendiente
SumoEnv._validate_action()	—	—	Pendiente
SumoEnv._get_state()	—	—	Pendiente

SumoEnv._get_traffic_data()	—	—	Pendiente
SumoEnv.reset()	—	—	Pendiente
CC promedio del módulo	—	—	Pendiente

agent/dqn_agent.py

Función / Método	CC	Clasificación	Acción
DQNAgent.select_action()	—	—	Pendiente
DQNAgent.train_step()	—	—	Pendiente
DQNAgent._compute_loss()	—	—	Pendiente
DQNAgent.update_target_network()	—	—	Pendiente
DQNAgent.save_checkpoint()	—	—	Pendiente
DQNAgent.load_checkpoint()	—	—	Pendiente
CC promedio del módulo	—	—	Pendiente

training/trainer.py

Función / Método	CC	Clasificación	Acción
Trainer.train()	—	—	Pendiente
Trainer._run_episode()	—	—	Pendiente
Trainer._decay_epsilon()	—	—	Pendiente
Trainer.evaluate()	—	—	Pendiente
Trainer.save_checkpoint()	—	—	Pendiente
CC promedio del módulo	—	—	Pendiente

environment/emissions.py

Función / Método	CC	Clasificación	Acción
EmissionCalculator.calculate()	—	—	Pendiente
EmissionCalculator._vsp_model()	—	—	Pendiente
EmissionCalculator._copert_model()	—	—	Pendiente
CC promedio del módulo	—	—	Pendiente

2.4 Resumen Global de Complejidad Ciclomática

Completar con: radon cc . -a

Módulo	CC promedio	CC máxima	Funciones CC > 10	Estado
config/config_loader.py	—	—	—	Pendiente
environment/sumo_env.py	—	—	—	Pendiente
environment/emissions.py	—	—	—	Pendiente
environment/reward.py	—	—	—	Pendiente
agent/dqn_agent.py	—	—	—	Pendiente
agent/replay_buffer.py	—	—	—	Pendiente
agent/network.py	—	—	—	Pendiente
training/trainer.py	—	—	—	Pendiente
training/baselines.py	—	—	—	Pendiente
evaluation/logger.py	—	—	—	Pendiente
evaluation/visualizer.py	—	—	—	Pendiente
PROYECTO COMPLETO	—	—	—	Pendiente

2.5 Criterios de Aceptación — Complejidad Ciclomática

Criterio	Umbral	Estado
CC promedio global del proyecto	≤ 8	Pendiente
Funciones con CC > 15	0 funciones	Pendiente
Funciones con CC > 10 en módulos críticos	≤ 2 funciones	Pendiente
CC máxima en cualquier función	≤ 20	Pendiente

2.6 Funciones con Mayor Riesgo Esperado

Las siguientes funciones tienen mayor probabilidad de CC elevada por su naturaleza:

Función	Razón de CC esperada alta	CC estimada
SumoEnv.step()	Maneja validación de acción, extracción de datos, cálculo de recompensa y detección de fin de episodio	8 — 12

Trainer.train()	Loop principal con múltiples condiciones de control: epsilon, checkpoint, early stopping	8 — 12
DQNAgent.train_step()	Condiciones de buffer mínimo, cálculo de pérdida, actualización del target	6 — 10
SumoEnv._validate_action()	Múltiples condiciones de seguridad vial: verde mínimo, amarillo, secuencia	6 — 9

2.7 Comando de Ejecución

bash

Reporte completo con clasificación por función

radon cc . -a -s --min A

Solo funciones con CC > 5

radon cc . -a -s --min B

Exportar a JSON

radon cc . -a -s -j > results/radon_cc_report.json

3. MÉTRICAS DE MANTENIBILIDAD

3.1 Índice de Mantenibilidad (MI)

El Índice de Mantenibilidad combina complejidad ciclomática, líneas de código y volumen de Halstead en un índice de 0 a 100.

MI	Clasificación	Interpretación
85 — 100	Verde	Fácil de mantener
65 — 84	Amarillo	Mantenibilidad moderada
0 — 64	Rojo	Difícil de mantener

3.2 Resultados de Índice de Mantenibilidad

Completar tras ejecutar: *radon mi . -s*

Módulo	MI	Clasificación	Estado
config/config_loader.py	—	—	Pendiente

environment/sumo_env.py	—	—	Pendiente
environment/emissions.py	—	—	Pendiente
environment/reward.py	—	—	Pendiente
agent/dqn_agent.py	—	—	Pendiente
agent/replay_buffer.py	—	—	Pendiente
agent/network.py	—	—	Pendiente
training/trainer.py	—	—	Pendiente
training/baselines.py	—	—	Pendiente
evaluation/logger.py	—	—	Pendiente
evaluation/visualizer.py	—	—	Pendiente

3.3 Criterio de Aceptación — Mantenibilidad

Criterio	Umbral	Estado
Módulos con MI en zona Roja (< 65)	0 módulos	Pendiente
Módulos con MI en zona Amarilla (65-84)	≤ 3 módulos	Pendiente
MI promedio global	≥ 70	Pendiente

3.4 Comando de Ejecución

bash

radon mi . -s --min B

4. MÉTRICAS DE DENSIDAD DE DEFECTOS

4.1 Definición

La densidad de defectos mide el número de defectos encontrados por unidad de código:

Densidad de defectos = Número de defectos / KLOC

(defectos por cada 1.000 líneas de código)

4.2 Fuentes de Defectos Consideradas

Fuente	Tipo de defecto
Bugs reportados en pruebas	Defectos funcionales detectados durante las pruebas

Issues críticos de pylint (E)	Defectos potenciales detectados por análisis estático
Vulnerabilidades bandit HIGH	Defectos de seguridad detectados por análisis estático
Errores de tipo mypy	Defectos de tipo detectados por análisis estático
Bugs reportados post-release	Defectos encontrados tras la liberación de cada versión

4.3 Tabla de Defectos por Sprint

Completar al cierre de cada sprint

Sprint	LOC desarrolladas	Defectos encontrados en pruebas	Defectos análisis estático	Total defectos	Densidad (def/KLOC)
Sprint 1	—	—	—	—	—
Sprint 2	—	—	—	—	—
Sprint 3	—	—	—	—	—
Sprint 4	—	—	—	—	—
TOTAL v1.0.0	—	—	—	—	—

4.4 Tabla de Defectos por Módulo

Completar al cierre del proyecto

Módulo	LOC	Defectos totales	Densidad (def/KLOC)	Clasificación
config/config_loader.py	—	—	—	Pendiente
environment/sumo_env.py	—	—	—	Pendiente
environment/emissions.py	—	—	—	Pendiente
environment/reward.py	—	—	—	Pendiente
agent/dqn_agent.py	—	—	—	Pendiente
agent/replay_buffer.py	—	—	—	Pendiente
agent/network.py	—	—	—	Pendiente
training/trainer.py	—	—	—	Pendiente
training/baselines.py	—	—	—	Pendiente
evaluation/logger.py	—	—	—	Pendiente

evaluation/visualizer.py	—	—	—	Pendiente
TOTAL	—	—	—	Pendiente

4.5 Clasificación de Densidad de Defectos

Densidad (def/KLOC)	Clasificación	Referencia industria
0 — 1.0	Excelente	Proyectos de alta calidad (NASA, sistemas críticos)
1.0 — 5.0	Buena	Software comercial bien desarrollado
5.0 — 10.0	Aceptable	Software comercial promedio
10.0 — 20.0	Regular	Requiere mejora de procesos
> 20.0	Inaceptable	Proceso de desarrollo con problemas graves

4.6 Criterio de Aceptación — Densidad de Defectos

Criterio	Umbral	Estado
Densidad global del proyecto v1.0.0	≤ 5.0 def/KLOC	Pendiente
Módulos con densidad > 10 def/KLOC	0 módulos	Pendiente
Defectos de seguridad HIGH en producción	0	Pendiente

5. MÉTRICAS DE TAMAÑO Y VOLUMEN

5.1 Líneas de Código por Módulo

Completar tras ejecutar: `radon raw . -s`

Módulo	LOC totales	LOC de código	Líneas de comentario	Líneas en blanco	% comentarios
config/config_loader.py	—	—	—	—	—
environment/sumo_env.py	—	—	—	—	—
environment/emissions.py	—	—	—	—	—
environment/reward.py	—	—	—	—	—
agent/dqn_agent.py	—	—	—	—	—
agent/replay_buffer.py	—	—	—	—	—
agent/network.py	—	—	—	—	—
training/trainer.py	—	—	—	—	—

training/baselines.py	—	—	—	—	—
evaluation/logger.py	—	—	—	—	—
evaluation/visualizer.py	—	—	—	—	—
tests/ (total)	—	—	—	—	—
TOTAL	—	—	—	—	—

5.2 Criterios de Aceptación — Tamaño

Criterio	Umbral	Estado
Porcentaje de comentarios en módulos de producción	$\geq 15\%$	Pendiente
LOC promedio por función en módulos críticos	≤ 30 LOC	Pendiente
Funciones con más de 50 LOC	≤ 3 en todo el proyecto	Pendiente

5.3 Comando de Ejecución

bash

radon raw . -s

6. RESUMEN EJECUTIVO DE MÉTRICAS DE CALIDAD

Métrica	Herramienta	Umbral	Valor obtenido	Estado
Errores flake8	flake8	0	—	Pendiente
Puntuación pylint	pylint	$\geq 8.0/10$	—	Pendiente
Vulnerabilidades críticas	bandit	0 HIGH+HIGH	—	Pendiente
Errores de tipo	mypy	0	—	Pendiente
CC promedio global	radon cc	≤ 8	—	Pendiente
Funciones CC > 15	radon cc	0	—	Pendiente
MI promedio global	radon mi	≥ 70	—	Pendiente
Módulos MI en zona roja	radon mi	0	—	Pendiente
Densidad de defectos	Manual + estático	≤ 5 def/KLOC	—	Pendiente
Cobertura de pruebas	pytest-cov	$\geq 80\%$	—	Pendiente
Reliability SonarQube	SonarQube	Rating A	—	Pendiente

Security SonarQube	SonarQube	Rating A	—	Pendiente
--------------------	-----------	----------	---	-----------